



Anil Seth

The Anatomy of an Android X86 Installation

The Android x86 ISO is under 200 MB, making it an attractive small distribution to explore. The ISO is a live CD with an option to install. It is fun to use this as a platform to explore and learn what else can be done with it, without rebuilding from the source.

Booting starts with a boot loader like Grub or syslinux. Two parameters are mandatory – the Linux kernel and an initial RAM file system (*initrd.img*). There may be additional kernel options which are used as and when needed by examining */proc/cmdline*. The kernel passes control to an executable called *init* in *initrd*. The *init* executable is often a shell script, though not necessarily.

This script will create a usable system—finding critical devices and mounting the real root file system. It will finally switch root to the real root and start another *init* script/ executable. In case the executable is not called *init*, it may be passed as a parameter to an option called *init*.

Let us now explore the *initrd.img* on the Android x86 ISO (<http://www.android-x86.org/>) and see how it works and what we can do with it. Extract the contents of the *initrd.img* into a working directory.

```
$sudo mount /repos/android-x86-4.2-20130228.iso /mnt/
livecd/
$cp /mnt/livecd/initrd.img initrd.gz
$zcat initrd.gz | cpio -ivd
```

The *Init* script is about 200 lines and easily readable. You can get an insight into what to look for by checking out the contents of *isolinux.cfg* in */mnt/livecd/isolinux*:

```
label livem
  menu label Live CD - ^Run Android-x86 without
  installation
    kernel /kernel
    append initrd=/initrd.img root=/dev/ram0 androidboot.
    hardware=android-x86 video=-16 quiet SRC= DATA=
```

Two unusual parameters are SRC and DATA. The *init* script searches for a root device by mounting each available disk partition and DVD, and searches for *ramdisk.img* and *system.sfs* in \$SRC. So, you can copy the contents of */mnt/*

livecd into */android* in the disk partition containing */boot*. Next, add the following entry in *grub.cfg*— preferably by editing *40_custom* in *grub.d* and regenerating *grub.cfg*.

```
menuentry 'Android '{
  linux /android/kernel root=/dev/ram0 androidboot.
  hardware=android-x86 video=-16 quiet SRC=/android DATA=
  initrd /android/initrd.img
}
```

You should now be able to boot into Android from the local disk using grub. In my experiments, Android booted on a netbook but not on a desktop. This is because there are no drivers in *initrd*. The kernel is supposed to include the required drivers for accessing the core devices. In fact, the *init* script has the option to boot from an NFS disk, provided the kernel is customised to include the required Ethernet and nfs drivers. The pre-built kernel is intended for use with common netbooks and small screen x86 devices.

Creating a directory */android/data* allows persistent storage. Alternatively, DATA can be assigned a block device.

Booting over the network

The next experiment would be to find out if you can boot over the network without customising the kernel. Can you follow the example of Puppy Linux and copy the necessary files inside *initrd.img*?

The *init* script mounts a disk partition on */mnt* and searches for the files needed. You can modify the script so that if it finds the files under */mnt*, it will not mount the partition.

On the DHCP, PXE boot server:

1. Create a directory *android* in *html-root*: */var/www/html* (Fedora) or */var/www* (Ubuntu). Use http rather than tftp as that is faster.
2. Copy the contents of */mnt/livecd* in it.
3. To enable pxe boot: